Basics Day 4 — Session 4 (Hours 13–16)

Python Programming (Basic) • Lists & Iteration

Session 4 Overview

- Hour 13: Lists fundamentals — create, index, slice, mutate
- Hour 14: Iterating lists with `for` loops — accumulators & averages
- Hour 15: Nested lists for table-like data — rows & columns
- Hour 16: Checkpoint 2 — Lists + string handling

Hour 13: Lists Fundamentals

Learning Outcomes

- Explain what a list is in plain, beginner-friendly language
- Create a list literal and print it
- Access items with indexing (including negative indices)
- Retrieve a portion of a list with slicing
- Mutate a list with `append()`, `remove()`, and `pop()`
- Check whether an item is present using the `in` operator

The Problem Lists Solve

One value at a time — the old way

```
item_1 = "milk"
item_2 = "bread"
item_3 = "eggs"
item_4 = "tea"
item_5 = "rice"
```

Why this breaks down

- Repetitive and hard to extend
- Difficult to print neatly
- Harder to search or update

> 💡 What if you suddenly need 6 items instead of 5?

What a List Is

One variable — many values

```
# A list holds multiple related values in order
groceries = ["milk", "bread", "eggs", "tea", "rice"]

print(groceries)
# ['milk', 'bread', 'eggs', 'tea', 'rice']
```

Key ideas

- Uses square brackets `[ ]`
- Items separated by commas
- Items stay in the order you put them
- One list = one variable name

> 💡 **Prediction:** What does `type(groceries)` return?

Creating Lists

A list with items

```
fruits = ["apple", "banana", "cherry"]
numbers = [10, 20, 30, 40, 50]
mixed = ["Alice", 25, True, 3.14]
```

An empty list (ready to fill later)

```
empty = []
shopping = []
```

How many items?

```
fruits = ["apple", "banana", "cherry"]
print(len(fruits))   # 3
```

Indexing — Finding Items by Position

Zero-based counting

```
fruits = ["apple", "banana", "cherry"]

fruits[0]   # "apple"   ← first item
fruits[1]   # "banana"  ← second item
fruits[2]   # "cherry"  ← third item
```

Negative indices — count from the end

```
fruits[-1]  # "cherry"  ← last item
fruits[-2]  # "banana"  ← second to last
```

> ⚠️ Lists start at index **0**, not **1**!

> 🗒️ **Speaker note:** Pause and ask: "What does `fruits[-1]` return?" Let learners predict before running it.

Slicing — Grabbing a Portion

The `[start:stop]` pattern

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]

fruits[0:2]   # ["apple", "banana"]      ← stop is excluded
fruits[1:4]   # ["banana", "cherry", "date"]
fruits[2:]    # ["cherry", "date", "elderberry"]  ← to end
fruits[:3]    # ["apple", "banana", "cherry"]     ← from start
```

Quick rule

- **start** is included
- **stop** is excluded (stop - 1 is the last item returned)

> 💡 This is the same rule as `range()` — you've seen it before!

Mutating a List — append() and update

Add an item to the end

```
groceries = ["milk", "bread"]

groceries.append("eggs")
print(groceries)  # ['milk', 'bread', 'eggs']
```

Update an item by index

```
groceries[0] = "oat milk"
print(groceries)  # ['oat milk', 'bread', 'eggs']
```

Lists are mutable — strings are not

```
name = "Alice"
name[0] = "B"   # ❌ TypeError — strings are immutable!
```

Removing Items — remove() and pop()

`remove()` — find and delete by value

```
fruits = ["apple", "banana", "cherry"]

fruits.remove("banana")
print(fruits)   # ['apple', 'cherry']
```

`pop()` — remove by index, returns the item

```
fruits = ["apple", "banana", "cherry"]

last = fruits.pop()     # removes last → "cherry"
first = fruits.pop(0)   # removes at index 0 → "apple"
print(fruits)           # ['banana']
```

Membership Test — the `in` Operator

Check if an item is in the list

```
fruits = ["apple", "banana", "cherry"]

"apple" in fruits    # True
"grape" in fruits    # False
"banana" not in fruits  # False
```

A practical use: check before removing

```
item = input("What would you like to remove? ")

if item in groceries:
    groceries.remove(item)
    print(f"Removed: {item}")
else:
    print(f"{item} was not in the list.")
```

Demo: Shopping List

Watch for…

- Starting with an empty list
- Appending items one by one
- Updating an item by index
- Removing by value safely
- Popping the last item

```
cart = []

cart.append("milk")
cart.append("bread")
cart.append("eggs")
cart.append("tea")
cart.append("rice")
```

Lab: Shopping List

Time: 25–35 minutes

Tasks

1. Start with an **empty list** called `shopping`
2. Use `append()` to add **5 items** (any items you like)
3. Print the list and its length after appending
4. Ask the user: "Which item would you like to remove?"
5. Check if it's in the list before removing — no crashes!
6. Print the **final list** and **final count**

Completion Criteria

✓ Empty list created with `[]`
✓ Exactly 5 items appended with `append()`

Common Pitfalls (Hour 13)

⚠️ **`remove()` on a missing item** — raises `ValueError`; always guard with `in`

⚠️ **`pop()` uses an index, not a value** — `fruits.pop("apple")` fails; use `fruits.remove("apple")`

⚠️ **Index starts at 0** — `fruits[1]` is the *second* item, not the first

⚠️ **Slice stop is excluded** — `fruits[0:2]` returns items at index 0 and 1 only

⚠️ **Confusing `append()` with `+`** — `list + item` raises a `TypeError`; use `list.append(item)`

```
# ❌ Common mistake
fruits = ["apple", "banana"]
fruits.remove("grape")    # ValueError: not in list

# ✓ Safe approach
if "grape" in fruits:
```

Quick Check (Hour 13)

Question: What does `fruits[-1]` return when `fruits = ["apple", "banana", "cherry"]`?

Expected Answer: `"cherry"` — the **last item** in the list.

Negative indices count from the end: `-1` → last, `-2` → second to last.

Hour 14: Iterating Lists with `for` Loops

Learning Outcomes

- Explain what `for item in items:` does in everyday language
- Loop through a list and perform an action for each item
- Use an accumulator variable to compute a running total
- Calculate an average from a list of numbers
- Find the highest value in a list with `max()`
- Build a grade-average program collecting 5 inputs

Why Loops Matter

The repetitive way — 5 grades, 5 print lines

```
grades = [88, 92, 76, 95, 84]

print(grades[0])
print(grades[1])
print(grades[2])
print(grades[3])
print(grades[4])
```

The problem: it doesn't scale

- 100 grades → 100 lines
- 1,000 grades → 1,000 lines
- If the list changes, you must rewrite everything

> 💡 We need one instruction that works for *any* list size.

The for Loop — Reading It in Plain English

Syntax

```
for item in items:
    # do something with item
```

Plain English translation

> "For **each** item in the items list,
> run the indented code once,
> using that item's value."

```
colors = ["red", "blue", "green"]

for color in colors:
    print(color)
```

Loop Variable Naming

The variable name is up to you

```
# These all do the same thing:
for color in colors:
    print(color)

for c in colors:
    print(c)

for x in colors:
    print(x)
```

Convention: singular = item, plural = list

```
for grade in grades:       # clear ✓
for student in students:   # clear ✓
for x in grades:           # works, but less readable
```

The Accumulator Pattern

Concept: keep a running total

```
grades = [88, 92, 76, 95, 84]

total = 0                   # ← start at zero (the accumulator)

for grade in grades:
    total += grade          # ← add each grade to total

print(total)                # 435
```

Step-by-step trace

Loop step	`grade`	`total`
Start	—	0
Step 1	88	88

Computing an Average

Total ÷ count

```
grades = [88, 92, 76, 95, 84]

total = 0
for grade in grades:
    total += grade

average = total / len(grades)   # 435 / 5 = 87.0

print(f"Average grade: {average:.1f}")
```

Using built-in max() and min()

```
highest = max(grades)   # 95
lowest = min(grades)    # 76

print(f"Highest: {highest}")
```

Demo: Grade Summary

Watch for…

- Accumulator starting at zero
- `total += grade` inside the loop body
- Division using `len(grades)` — not a hard-coded number
- `max()` without writing a manual loop

```
grades = [88, 92, 76, 95, 84]

total = 0
for grade in grades:
    total += grade

average = total / len(grades)
highest = max(grades)

print(f"Grades:  {grades}")
```

Collecting Input into a List

Asking for values one at a time

```
grades = []

grades.append(int(input("Enter grade 1: ")))
grades.append(int(input("Enter grade 2: ")))
grades.append(int(input("Enter grade 3: ")))
grades.append(int(input("Enter grade 4: ")))
grades.append(int(input("Enter grade 5: ")))

print(f"Collected: {grades}")
```

> ⚠️ Don't forget `int()` — `input()` always returns a **string**!

Lab: Grade Average

Time: 25–35 minutes

Tasks

1. Create an **empty list** called `grades`
2. Ask the user to enter **5 numeric grades** — append each to the list
3. Compute the **total** using an accumulator loop
4. Compute the **average** using `len(grades)`
5. Find the **highest grade** with `max()`
6. Print a clean summary report

Completion Criteria

✓ All 5 inputs converted to `int` or `float`
✓ Accumulator starts at `0` before the loop

Common Pitfalls (Hour 14)

⚠️ **Forgetting `int()` or `float()` on `input()`** — `"88" + "92"` = `"8892"`, not `180`

⚠️ **Accumulator not initialized before the loop** — `total` must be `0` before the `for` statement

⚠️ **Dividing by a hard-coded count** — use `len(grades)` so it works for any list size

⚠️ **Indentation errors** — the `total += grade` line must be *inside* the loop (4 spaces in)

```
# ❌ Indentation error — total not updated inside loop
total = 0
for grade in grades:
    pass
total += grade    # runs once, uses only the last value

# ✓ Correct
```

Quick Check (Hour 14)

Question: What does an accumulator pattern look like for summing a list?

Expected Answer:

```
total = 0
for x in nums:
    total += x
```

Start the accumulator at `0` *before* the loop.

Add each item to it *inside* the loop.

Read the result *after* the loop ends.

Hour 15: Nested Lists for Table-Like Data

Learning Outcomes

- Explain what a nested list is in simple language
- Represent small table-like data as a list of lists
- Access a cell using `grid[row][col]` notation
- Print each row of a nested list with a `for` loop
- Update one value inside a 2D structure
- Avoid off-by-one row/column mistakes

From One Row to a Grid

Single-row list (what we know)

```
row = ["A1", "A2", "A3"]
```

Grid — a list where each item is another list

```
seats = [
    ["A1", "A2", "A3"],   # row 0
    ["B1", "B2", "B3"],   # row 1
    ["C1", "C2", "C3"],   # row 2
]
```

Mental model

- The **outer** list holds the rows
- Each **inner** list holds the columns for that row

When Do You Need a Grid?

Real examples of table-like data

- 🪑 Classroom seating chart
- ✅ Tic-tac-toe or noughts-and-crosses board
- 📅 Weekly schedule (rows = days, cols = periods)
- 📊 Simple spreadsheet of student scores

Why not a flat list?

```
# Flat — structure is hidden
seats_flat = ["A1", "A2", "A3", "B1", "B2", "B3"]

# Nested — structure is visible
seats = [["A1", "A2", "A3"], ["B1", "B2", "B3"]]
```

> 💡 A nested list makes the *shape* of the data visible in the code.

Accessing Cells: grid[row][col]

Two-step indexing

```
seats = [
    ["A1", "A2", "A3"],
    ["B1", "B2", "B3"],
    ["C1", "C2", "C3"],
]

seats[0]        # ["A1", "A2", "A3"]   ← whole row 0
seats[0][1]     # "A2"                 ← row 0, col 1
seats[1][2]     # "B3"                 ← row 1, col 2
seats[2][0]     # "C1"                 ← row 2, col 0
```

Visual grid map

```
        col 0   col 1   col 2
row 0 [ "A1",  "A2",  "A3" ]
row 1 [ "B1",  "B2",  "B3" ]
```

Updating a Cell

Change one seat to "X" (reserved)

```
seats = [
    ["A1", "A2", "A3"],
    ["B1", "B2", "B3"],
    ["C1", "C2", "C3"],
]

seats[1][2] = "X"   # row 1, col 2  →  "B3" → "X"

print(seats[1])     # ['B1', 'B2', 'X']
```

The assignment works just like a regular list

```
# Single list:  items[index] = new_value
# Nested list:  grid[row][col] = new_value
```

> ⚠️ Remember: both indices are **zero-based**!

Printing Rows with a for Loop

Print each row on its own line

```
seats = [
    ["A1", "A2", "A3"],
    ["B1", "B2", "B3"],
    ["C1", "C2", "C3"],
]

for row in seats:
    print(row)
```

```
['A1', 'A2', 'A3']
['B1', 'B2', 'B3']
['C1', 'C2', 'C3']
```

Reading the loop in plain English

> "For **each row** in seats, print that row."

Demo: Seating Chart — Create, Print, Update

Watch for…

- Three rows defined as a list of lists
- `seats[row][col]` access syntax
- Updating one cell and reprinting to verify

```
seats = [
    ["A1", "A2", "A3"],
    ["B1", "B2", "B3"],
    ["C1", "C2", "C3"],
]

print("Original seating chart:")
for row in seats:
    print(row)

# Reserve seat B2 (row 1, col 1)
seats[1][1] = "X"
```

Lab: Seating Chart

Time: 25–35 minutes

Tasks

1. Create a **3×3 seating chart** as a nested list using any labels you like
2. **Print the full grid** row by row before any changes
3. Ask the user for a **row number** (0–2) and a **column number** (0–2)
4. **Change that seat** to `"X"` (reserved)
5. **Print the grid again** to confirm the change

Completion Criteria

✓ Grid defined as a list of 3 lists, each with 3 string items
✓ `for row in seats: print(row)` used for both prints
✓ User input converted to `int()` before use as index

Common Pitfalls (Hour 15)

⚠️ **Swapping row and col** — `seats[col][row]` gives wrong cell; always row first, then column

⚠️ **Forgetting zero-based counting** — `seats[3][0]` on a 3-row grid → `IndexError`

⚠️ **Not converting input to `int()`** — `seats["1"]["2"]` raises a `TypeError`

⚠️ **Off-by-one on update** — want row 2 col 1 → use `seats[2][1]`, not `seats[3][2]`

```
# ❌ Swapped indices (common beginner mistake)
seats[col][row] = "X"     # wrong order

# ✓ Correct: row first, then column
seats[row][col] = "X"

# ❌ Forgetting int() on input
row = input("Row: ")        # row is a string!
seats[row][0]               # TypeError
```

Quick Check (Hour 15)

Question: How do you access row 1, column 2 of a grid called `seats`?

Expected Answer: `seats[1][2]`

- First index = **row** (outer list position)
- Second index = **column** (inner list position)
- Both are **zero-based**

Hour 16: Checkpoint 2 — Lists + String Handling

Learning Outcomes

- Demonstrate list creation and updates in a short checkpoint program
- Use strings and list operations together for clean user-facing output
- Display a numbered list of items using `enumerate()`
- Add items, remove items, and show items in a simple in-memory to-do list
- Explain the difference between `remove()` (by value) and `pop()` (by index)
- Produce correct, readable, crash-free output

Checkpoint 2 — Framing

What this checkpoint tests

- Can you store items in a list?
- Can you add and remove items correctly?
- Can you display items in a numbered, readable format?
- Does your program handle a missing item gracefully?

What this checkpoint does NOT require

- ✗ Looping menus (while loops) — not yet
- ✗ Dictionaries or files
- ✗ Functions or classes
- ✗ Any Advanced-module topics

> 💡 One clean pass through the user's chosen action is all that's needed.

Checkpoint Scope Guardrail

✓ What belongs in this checkpoint

- `list = []` — list creation
- `list.append(item)` — adding items
- `list.remove(item)` — removing by value
- `item in list` — checking before removing
- `enumerate()` — numbered display
- `input()`, `if/elif/else`, f-strings

✗ Out of scope — keep for later

- `while True:` looping menu
- Dictionaries (`{}`)
- File reading/writing

Review: The Three Core List Actions

Add an item

```
todo_items = []
new_item = input("Enter item to add: ")
todo_items.append(new_item)
print(f"Added: '{new_item}'")
```

Remove an item (with safety check)

```
item_to_remove = input("Item to remove: ")
if item_to_remove in todo_items:
    todo_items.remove(item_to_remove)
    print(f"Removed: '{item_to_remove}'")
else:
    print(f"'{item_to_remove}' was not found in the list.")
```

Show all items

Numbered Output with enumerate()

The pattern

```
items = ["study Python", "buy groceries", "call mom"]

for i, item in enumerate(items, 1):
    print(f"{i}. {item}")
```

Output

```
1. study Python
2. buy groceries
3. call mom
```

What `enumerate(items, 1)` gives you

- `i` — the counter (starts at `1` because we passed `1`)
- `item` — the current list item

The Full Checkpoint Program

```python
# Pre-filled list (lab requires 3 starting items)
todo_items = ["buy milk", "send email", "read notes"]

action = input("Choose: add, remove, or show: ")

if action == "add":
    new_item = input("Enter item: ")
    todo_items.append(new_item)
    print(f"Added: '{new_item}'")

elif action == "remove":
    item_to_remove = input("Item to remove: ")
    if item_to_remove in todo_items:
        todo_items.remove(item_to_remove)
        print(f"Removed: '{item_to_remove}'")
    else:
        print(f"'{item_to_remove}' not found.")

elif action == "show":
    if len(todo_items) == 0:
        print("Your to-do list is empty.")
    else:
        for i, item in enumerate(todo_items, 1):
            print(f"{i}. {item}")

else:
    print("Unrecognized action. Choose: add, remove, or show.")
```

Demo: To-Do List in Action

Watch for…

- Pre-filled list as the starting point
- `in` guard before `remove()`
- `enumerate(items, 1)` for numbered output
- Clean branch separation with `if/elif/elif`

```
todo_items = ["email team", "buy notebook", "study lists"]

# Simulate: show action
print("Current to-do list:")
for i, item in enumerate(todo_items, 1):
    print(f"{i}. {item}")

# Simulate: remove action
to_remove = "buy notebook"
if to_remove in todo_items:
```

Checkpoint Lab: Simple To-Do List

Time: 25–35 minutes

Task

Build a simple in-memory to-do list program. A single run of the program should:

1. Start with a **pre-filled list** of 3 to-do items
2. Ask the user to choose an action: `add`, `remove`, or `show`
3. For **add**: ask for a new item, append it, confirm it was added
4. For **remove**: ask for an item, check it exists, remove it, confirm
5. For **show**: display items with **numbered output** using `enumerate()`
6. Handle an unrecognized action with a friendly message

Completion Criteria

remove() vs pop() — Final Clarity

`remove()` — by value

```
items = ["walk dog", "read book", "cook dinner"]

items.remove("read book")   # finds the value and removes it
print(items)
# ['walk dog', 'cook dinner']
```

`pop()` — by index

```
items = ["walk dog", "read book", "cook dinner"]

removed = items.pop(1)      # removes at index 1
print(removed)              # "read book"
print(items)
# ['walk dog', 'cook dinner']
```

When to use which

Common Pitfalls (Hour 16)

⚠️ **Calling `remove()` without an `in` guard** — raises `ValueError` if item is missing

⚠️ **Using `pop("item")` instead of `remove("item")`** — `pop()` needs an index, not a value

⚠️ **`enumerate()` starting at 0** — pass `1` as second argument for human-friendly numbering

⚠️ **Forgetting to test all three branches** — test add, remove, and show before calling done

```
# ❌ Crashes if item not found
items.remove("missing item")   # ValueError

# ✓ Safe
if "missing item" in items:
    items.remove("missing item")
```

Quick Check (Hour 16)

Question: How do you display a numbered list of items from a list called `todo_items`?

Expected Answer:

```
for i, item in enumerate(todo_items, 1):
    print(f"{i}. {item}")
```

- `enumerate(todo_items, 1)` gives you both the **counter** (`i`) and the **item**
- Starting at `1` makes the numbering human-friendly

Session 4 Wrap-Up

What We Covered Today

Hour 13: Lists Fundamentals

- List literals, indexing (0-based), negative indices
- Slicing with `[start:stop]`
- Mutating with `append()`, `remove()`, `pop()`
- Membership test with `in`

Hour 14: Iterating Lists with `for` Loops

- `for item in items:` — plain English mental model
- Accumulator pattern: `total = 0` → `total += item`
- Average: `total / len(items)`

Scope Guardrail Reminder

Stay in Basics Scope ✓

✓ List creation, indexing, slicing
✓ `append()`, `remove()`, `pop()`, `in`
✓ `for item in items:` loop
✓ Accumulator pattern for totals and averages
✓ Nested lists: `grid[row][col]`
✓ `enumerate()` for numbered display

Not Yet (Advanced Topics) ✗

✗ Dictionaries (`{key: value}`)
✗ File I/O (`open()`, `read()`, `write()`)
✗ Functions (`def`) and return values
✗ List comprehensions

No-Go Topics for Basics Course

Keep for Advanced Module

- Web frameworks (Flask/Django)
- Databases and SQL
- GUI frameworks (Tkinter/PyQt)
- Testing frameworks (pytest)
- Packaging and distribution
- Decorators and generators
- Lambda functions
- Advanced OOP patterns
- Regular expressions

Homework / Practice

Recommended Exercises

1. **Extend the shopping list** — add a `show all` action that prints items numbered with `enumerate()`
2. **Grade tracker** — collect 5 grades, print average, highest, and lowest using `max()` / `min()`
3. **Tic-tac-toe board** — create a 3×3 nested list of `"."` and let two users place their marks
4. **To-do list upgrade** — add a count display: `"You have 3 items remaining"`

Next Session Preview

Session 5 (Hours 17–20)

- Hour 17: Functions — defining and calling `def`
- Hour 18: Parameters, arguments, and return values
- Hour 19: Scope — local vs. global variables
- Hour 20: Checkpoint 3 — Functions mini-assessment

Questions?

Key patterns from today:

- `list[-1]` → last item
- `for item in items:` → visit each item once
- `total = 0; for x in nums: total += x` → accumulator
- `grid[row][col]` → two-index access
- `for i, item in enumerate(items, 1):` → numbered output

Thank You!

See you in Session 5!